

SUPERDEC: 3D Scene Decomposition with Superquadric Primitives

Elisabetta Fedele^{1,2} Boyang Sun¹ Leonidas Guibas² Marc Pollefeys^{1,3} Francis Engelmann²

¹ETH Zurich ²Stanford University ³Microsoft

Abstract

We present SUPERDEC, an approach for creating compact 3D scene representations via decomposition into superquadric primitives. While most recent methods use geometric primitives to obtain photorealistic 3D reconstructions, we instead leverage them to obtain a compact yet expressive representation. To this end, we design a novel architecture that efficiently decomposes point clouds of arbitrary objects into a compact set of superquadrics. We train our model on ShapeNet and demonstrate its generalization capabilities on object instances from ScanNet++ as well as on full Replica scenes. Finally, we show that our compact superquadric-based representation supports a wide range of downstream applications, including robotic manipulation and controllable visual content generation. Project page: <https://super-dec.github.io>.

1. Introduction

3D scene representations play a central role in computer vision and robotics, enabling tasks such as 3D scene understanding [34, 48, 49], scene generation [16, 42, 43], functional reasoning [20, 59], and scene interaction [9, 13, 22, 39]. Recent work [19] utilizes 3D Gaussians as geometric primitives to produce high-quality, photorealistic reconstructions. However, such representations are often memory-intensive. In contrast, we propose a more lightweight yet geometrically faithful 3D scene representation by decomposing the input point cloud into a compact set of explicit primitives, namely superquadrics (Fig. 1).

Representations for 3D scenes include well-established formats such as point clouds, meshes, signed distance functions, and voxel grids, each offering different trade-offs among geometric detail, computational cost, resolution, performance, interpretability, and editability. Recently, multi-view approaches such as Neural Radiance Fields (NeRF) [30] and Gaussian Splatting (GS) [19] have gained popularity as 3D scene representations. These methods optimize photometric losses to ensure that their underlying representations (implicit in the case of NeRF, and explicit in the case of GS) are consistent with observed images. While



Figure 1. **3D Scene Decomposition with Superquadrics.** Given a 3D point cloud of an indoor scene, SUPERDEC decomposes all scene objects into a compact set of superquadric primitives. Different object instances are visualized with different colors.

these approaches excel at achieving photorealism, they lack explicit control over compactness, often resulting in large, non-modular scene encodings that are unsuitable for tasks requiring explicit spatial reasoning.

While optimizing compactness on a scene level remains a challenging task, prior work has shown that geometric primitives such as cuboids [51, 57] or superquadrics [1, 32, 33] enable compact and interpretable decompositions of individual objects. These methods are generally either learning-based [33, 57], favoring speed at the cost of geometric accuracy, or optimization-based [1, 24, 32], offering improved accuracy at the expense of higher computational cost. Although both types of approaches can be effective for certain object classes, they often struggle to generalize across datasets with diverse object geometries. Learning-based methods typically require category-specific training, while optimization-based ones rely on hand-crafted heuristics, limiting their scalability in open-world settings.

Motivated by the abstraction capabilities of geometric primitives for individual object categories, we propose to represent complex 3D scenes using a compact set of superquadrics. To this end, we learn *class-agnostic* object-

level shape priors to optimize compactness and leverage an off-the-shelf 3D instance segmentation method (Mask3D [41]) to scale our approach to full 3D scenes.

We choose superquadrics as building blocks for our representation, as they offer more accurate shape modeling than cuboids while incurring minimal additional parameter overhead (9 v.s. 11, including 6-DoF pose parameters). To develop a model capable of generalizing across diverse object types, we draw inspiration from supervised segmentation [5, 6, 41], and approach the problem from the perspective of unsupervised geometric-based segmentation, using local point-based features to iteratively refine predicted geometric primitives. Our model is trained on ShapeNet [4] and evaluated on three challenging and diverse 3D datasets: ShapeNet [4], ScanNet++ [58], and Replica [45]. On the 3D object dataset ShapeNet, our approach achieves an L2 error six times smaller than prior state-of-the-art methods [33], while requiring only half the number of primitives. On ScanNet++ and Replica, we demonstrate that our method generalizes well to real-world scene-level settings, despite being trained solely on ShapeNet. Finally, we show the practical utility of our method as a scene representation for robotic tasks, including path planning and object grasping, as well as an editable 3D scene representation for controllable image generation.

In summary, our contributions are the following:

- 1) We introduce SUPERDEC, a novel method for decomposing 3D scenes using superquadric primitives.
- 2) SUPERDEC achieves state-of-the-art object decomposition scores on ShapeNet trained jointly on multiple classes.
- 3) We demonstrate the effectiveness of 3D superquadric scene representations for robotic tasks and controllable generative content creation.

2. Related Work

Learning-based methods have shown that neural networks, when equipped with suitable reconstruction losses, can directly predict geometric primitive parameters to decompose point clouds into a minimal set of primitives for specific object categories. Tulsiani [51] introduced a CNN-based method for cuboid decomposition, which was later extended to more expressive primitives such as superquadrics by Paschalidou *et al.* [33]. CSA [57] further enhanced interpretability by employing a stronger point encoder and jointly predicting cuboid parameters and part segmentations. However, these methods remain constrained by their reliance on category-specific training. We attribute this limitation to their model design, which encodes only global shape features, sufficient for intra-category generalization but ineffective for decomposing out-of-category objects.

Optimization-based methods largely originate from the literature on superquadric fitting. EMS [24] revisited this line of work by introducing a probabilistic formulation that

enables the decomposition of arbitrary objects into multiple superquadrics. Given an input point cloud, the method first fits a superquadric to the main structure and identifies unfitted outlier clusters, which are then recursively processed in a hierarchical fashion up to a predefined depth level. However, as noted in their paper and confirmed by our experiments, this approach implicitly assumes that objects exhibit a hierarchical geometric structure, limiting its applicability to many real-world objects such as tables and chairs. Other methods, such as Marching Primitives [25], require Signed Distance Functions (SDFs) as input, which are generally not easily available in real-world scenes. More fundamentally, since these approaches optimize from scratch for each object, they cannot leverage generalizable point features or learned shape priors, both of which are critical for abstraction and robustness under partial observations, a common challenge in practical 3D capture scenarios.

Scene-level decomposition. With the emergence of 3DGS [19], an increasing number of works have explored representing 3D scenes using various geometric primitives as generalized ellipsoids [15] and convexes [14]. While heuristics can control the number of Gaussians, achieving truly compact representations remains challenging. DBW [32] addresses this by fitting a small set of textured superquadrics to 3D scenes, building on the principles of 3DGS. Given a set of scene images, it performs test-time optimization with a photometric loss and renders the primitives using a differentiable rasterizer [23]. To model the environment, DBW adds a meshed ground plane and a meshed icosphere for the background. However, it is restricted to scenes with fewer than 10 primitives and requires objects to be aligned to a ground plane. Furthermore, the optimization is computationally expensive, taking around three hours even on simple DTU [18] scenes. Our method differs significantly in terms of input requirements, generality of application, and computational efficiency.

Superquadrics are a parametric family of shapes introduced by Barr *et al.* [2] in 1981 and have since been widely adopted in both computer vision and graphics [7, 35, 44]. Their popularity stems from their ability to represent a diverse range of shapes with a highly compact parameterization. A superquadric in its canonical pose is defined by just five parameters: (s_x, s_y, s_z) for the scales along the three principal semi-axes and (ϵ_1, ϵ_2) for the shape-defining exponents. Given those parameters, their surface is described by the implicit equation:

$$f(\mathbf{x}) = \left(\left(\frac{x}{s_x} \right)^{\frac{2}{\epsilon_2}} + \left(\frac{y}{s_y} \right)^{\frac{2}{\epsilon_2}} \right)^{\frac{\epsilon_2}{\epsilon_1}} + \left(\frac{z}{s_z} \right)^{\frac{2}{\epsilon_1}} = 1. \quad (1)$$

Extending this representation to a global coordinate system requires 6 additional parameters (3 for translation and 3

for rotation), resulting in a total of 11 parameters per superquadric. Another key property of superquadrics is the ability to compute the *radial distance* from any point in 3D space to the superquadric surface, i.e., the distance between a point and the superquadric’s surface along the line connecting that point to the center of the superquadric. Specifically, given a point $\mathbf{x} \in \mathbb{R}^3$, its radial distance to the surface of a canonically oriented superquadric is defined as:

$$d_r = |\mathbf{x}| \cdot |1 - f(\mathbf{x})^{-\epsilon_1/2}|, \quad (2)$$

where $f(\mathbf{x})$ is given in Eq. 1. We refer the reader to [53] for the derivation of Eq. 2 and to [17] for a more comprehensive overview on superquadrics.

3. Method

Our ultimate goal is a 3D scene decomposition using superquadric primitives. To this end, we first focus on single-object decomposition and then show how our method, combined with 3D instance segmentation [41], can be applied to full 3D scenes. We detail the single-object approach in Sec. 3.1 and its extension to full scenes in Sec. 3.2.

3.1. Single Object Decomposition

Fig. 2 illustrates our model for single-object decomposition. It consists of two main components: a self-supervised feed-forward neural network that jointly predicts superquadric parameters and a segmentation matrix associating points to superquadrics, followed by a lightweight Levenberg–Marquardt (LM) optimization [21, 29].

3.1.1. Feed-forward Neural Network

Our deep learning model draws inspiration from recent fully-supervised Transformer-based [54] segmentation models [5, 6, 41]. These models iteratively decode a sequence of queries, each representing a segmentation mask, by cross-attending to input pixels or points. In our case, the queries represent superquadrics. Next, we show how such an architecture can be adapted to *unsupervisedly* segment superquadrics, instead of *supervisedly* segment objects.

Model Details. Given an input point cloud $\mathcal{P} \in \mathbb{R}^{N \times 3}$, where each of the N points has a 3D coordinate, we first extract rich point features $\mathcal{F}_{PC} \in \mathbb{R}^{N \times H}$ using the PVCNN [26] point encoder. At the same time, we initialize P superquadrics features $\mathcal{F}_{SQ} \in \mathbb{R}^{P \times H}$ with sinusoidal positional encodings. We feed these features in a Transformer decoder [54] which leverages self-attention, cross-attention to the point features, and feed-forward layers to refine them. Once refined, the superquadric features $\mathcal{F}_{SQ}$ and the point features $\mathcal{F}_{PC}$ are fed into two prediction heads: The *segmentation head* takes as input $\mathcal{F}_{SQ}$ and $\mathcal{F}_{PC}$ and predicts a soft assignment matrix $M \in \mathbb{R}^{N \times P}$ associating points to

superquadrics and whose elements are defined as:

$$m_{ij} = \sigma(\phi(\mathcal{F}_{PC}) \cdot \mathcal{F}_{SQ}), \quad (3)$$

where $\phi(\mathcal{F}_{PC}) \in \mathbb{R}^{N \times H}$ is a learned projection of the point features to match the dimensionality of the superquadric features, and σ is the softmax function. The second head, the *superquadric head*, takes the superquadric features $\mathcal{F}_{SQ}$ as input and predicts 12 parameters for each superquadric: 11 encoding its 5-DoF shape and 6-DoF pose, and one modeling its existence probability α , enabling a variable number of superquadrics per object.

Losses. We train our model in a self-supervised manner, without requiring any ground truth annotation. Specifically, the total loss is defined as:

$$\mathcal{L} = \mathcal{L}_{rec} + \lambda_{par} \mathcal{L}_{par} + \lambda_{exist} \mathcal{L}_{exist}, \quad (4)$$

where $\mathcal{L}_{rec}$ is the reconstruction loss aligning the predicted superquadrics to the input point cloud $\mathcal{P}$, $\mathcal{L}_{par}$ is the parsimony loss encouraging a small number of primitives, $\mathcal{L}_{exist}$ is the existence loss, and λ_{par} , λ_{exist} are weighting coefficients. The reconstruction loss $\mathcal{L}_{rec}$ consists of three terms:

$$\mathcal{L}_{rec} = \mathcal{L}_{\mathcal{P} \rightarrow SQ} + \mathcal{L}_{SQ \rightarrow \mathcal{P}} + \mathcal{L}_N. \quad (5)$$

The first two terms correspond to the bi-directional Chamfer distance between the input point cloud and the superquadric surfaces, while the third term serves as a regularizer incorporating normal information to improve convergence during training. To compute the Chamfer distance, we approximate each superquadric surface by uniformly sampling S points, following the method of Pilu *et al.* [36]. Denoting by $d(\mathbf{x}_i, \mathbf{x}'_{js})$ the euclidean distance between the i -th point in the input point cloud and the s -th point sampled on the surface of the j -th superquadric, we define $\mathcal{L}_{\mathcal{P} \rightarrow SQ}$ as:

$$\mathcal{L}_{\mathcal{P} \rightarrow SQ} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^P m_{ij} \min_{s \in [S]} d(\mathbf{x}_i, \mathbf{x}'_{js}), \quad (6)$$

and $\mathcal{L}_{SQ \rightarrow \mathcal{P}}$ as:

$$\mathcal{L}_{SQ \rightarrow \mathcal{P}} = \frac{1}{S \sum_{j=1}^P \alpha_j} \sum_{j=1}^P \alpha_j \sum_{s=1}^S \min_{i \in [N]} d(\mathbf{x}_i, \mathbf{x}'_{js}). \quad (7)$$

The last term of Eq. 5, i.e., $\mathcal{L}_N$ is defined as the reconstruction loss from Yang *et al.* [57], and is used to incorporate normal information during training which leads to accelerated convergence. Additionally, since we seek not only accuracy but also compactness, we introduce a parsimony loss to encourage the use of fewer primitives. To do that, we optimize the 0.5-norm of $m_j := \sum_{i=1}^N \frac{m_{ij}}{N}$ and define the parsimony loss as:

$$\mathcal{L}_{par} = \left(\frac{1}{P} \sum_{j=1}^P \frac{\sqrt{m_j}}{P} \right)^2. \quad (8)$$

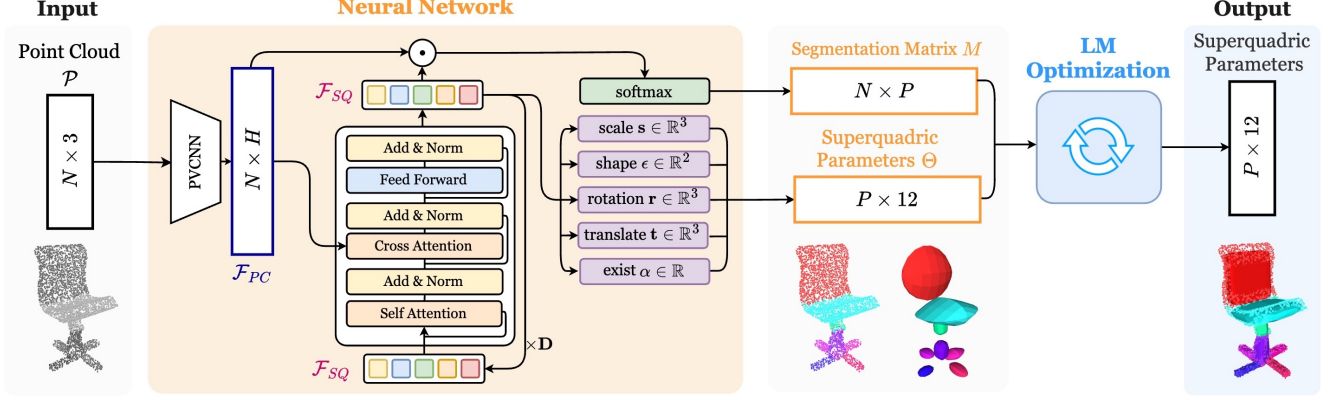


Figure 2. **Illustration of the SUPERDEC Model.** Given a point cloud of an object with N points, a Transformer-based neural network predicts parameters for P superquadrics, as well as a soft segmentation matrix that assigns points to superquadrics. The predicted parameters include the 11 superquadric parameters and an objectness score. These predictions provide an effective initialization for the subsequent Levenberg–Marquardt (LM) optimization, which refines the superquadrics.

Lastly, we employ an existence loss $\mathcal{L}_{exist}$ which uses the predicted segmentation as a teacher for the linear head in charge of predicting the existence probability. More specifically, given a threshold ϵ_{exist} , we define the ground-truth existence of the j th superquadric as $\hat{\alpha}_j := m_j > \epsilon_{exist}$ and define $\mathcal{L}_{exist}$ as:

$$\mathcal{L}_{exist} = \sum_{j=1}^P \frac{BCE(\alpha_j, \hat{\alpha}_j)}{P}, \quad (9)$$

where BCE is the binary cross entropy and α_j is the predicted existence probability for the j th superquadric.

3.1.2. Optimization

Our optimization module takes as input the predicted soft segmentation matrix M as well as the superquadric parameters Θ , and further refines the superquadric parameters using the Levenberg–Marquardt (LM) [21, 29] algorithm. Specifically, given a point cloud of N points, it iteratively refines the parameters Θ_j of the j th superquadric, by computing two sets of residuals: The first set of residuals r_{ij} with $i \in [1, N]$ and $j \in [1, P]$ is defined as:

$$r_{ij} = m_{ij} \tilde{d}_j(\mathbf{x}_i), \quad (10)$$

where $\tilde{d}_j(\mathbf{x}_i)$ denotes the radial distance of point $\mathbf{x}_i$ from the j th superquadric, computed according to Eq. 2. The second set of residuals is used for normalization and is obtained by sampling a set of K points $\mathbf{p}_1, \dots, \mathbf{p}_K$ on the surface of the given superquadric and then computing the distance of each of them from the point cloud. Specifically, for $i \in [N + 1, N + K]$ and $j \in [1, P]$ we compute r_{ij} as:

$$r_{ij} = \min_k \|\mathbf{p}_i - \mathbf{N} - \Pi_j(\mathbf{x}_k)\|_2, \quad \text{with } k \in [1, N]. \quad (11)$$

3.2. Decomposition of Full 3D Scenes

After training on single objects, extending SUPERDEC to full 3D scenes is straightforward. Given a scene-level point cloud, we extract 3D object instance masks using Mask3D [41]. Each object is centered and uniformly rescaled to a sphere of radius 0.5. We then predict the superquadric primitives for each object individually using our model. We found our model trained on ShapeNet [4] to generalize well on real-world 3D scenes from ScanNet++ [58] and Replica [45] without additional fine-tuning.

4. Experiments

We first compare our SUPERDEC with previous state-of-the-art methods on individual objects and full 3D scenes (Sec. 4.1). We then demonstrate the usefulness of our representation on down-stream applications for robotics and controllable image generation (Sec. 4.2). Finally, in Sec. 4.3, we present additional analyses on part segmentation and the implicit learning of shape categories, followed by a study of the compactness–accuracy trade-off and runtime.

4.1. Comparing with State-of-the-art Methods

Datasets. We compare on three different datasets: *ShapeNet* [4]: We use the 13 classes of the ShapeNet subset and train-val-test splits as defined in Choy *et al.* [8]. All objects are pre-aligned in a canonical orientation. For each object we sample 4096 points using Farthest Point Sampling (FPS) [37]. ShapeNet is a widely used dataset and is well-suited for comparison with existing baselines.

ScanNet++ [58]: We further evaluate our model on real-world object scans from the ScanNet++ validation set. Each object is extracted using ground truth mask annotations, and 4096 points per object are sampled with FPS. In contrast to ShapeNet, these object point clouds are noisier, partially ob-

Model	Primitive Type	Segmentation	In-category			Out-of-category		
			L1 ↓	L2 ↓	#Prim. ↓	L1 ↓	L2 ↓	#Prim. ↓
EMS (Liu <i>et al.</i>) [24]	Superquadrics	✗	5.771	1.345	5.68	5.410	1.211	5.68
CSA (Yang <i>et al.</i>) [57]	Cuboids	✓	5.157	0.527	9.21	4.897	0.427	11.75
SQ (Paschalidou <i>et al.</i>) [33]	Superquadrics	✗	3.668	0.279	10	4.193	0.354	9
SUPERDEC (Ours)	Superquadrics	✓	1.698	0.047	5.8	1.847	0.061	5.26

Table 1. **Quantitative Results on ShapeNet [4]**. We show scores for in-category and out-of-category experiments and are scaled by 10^2 .

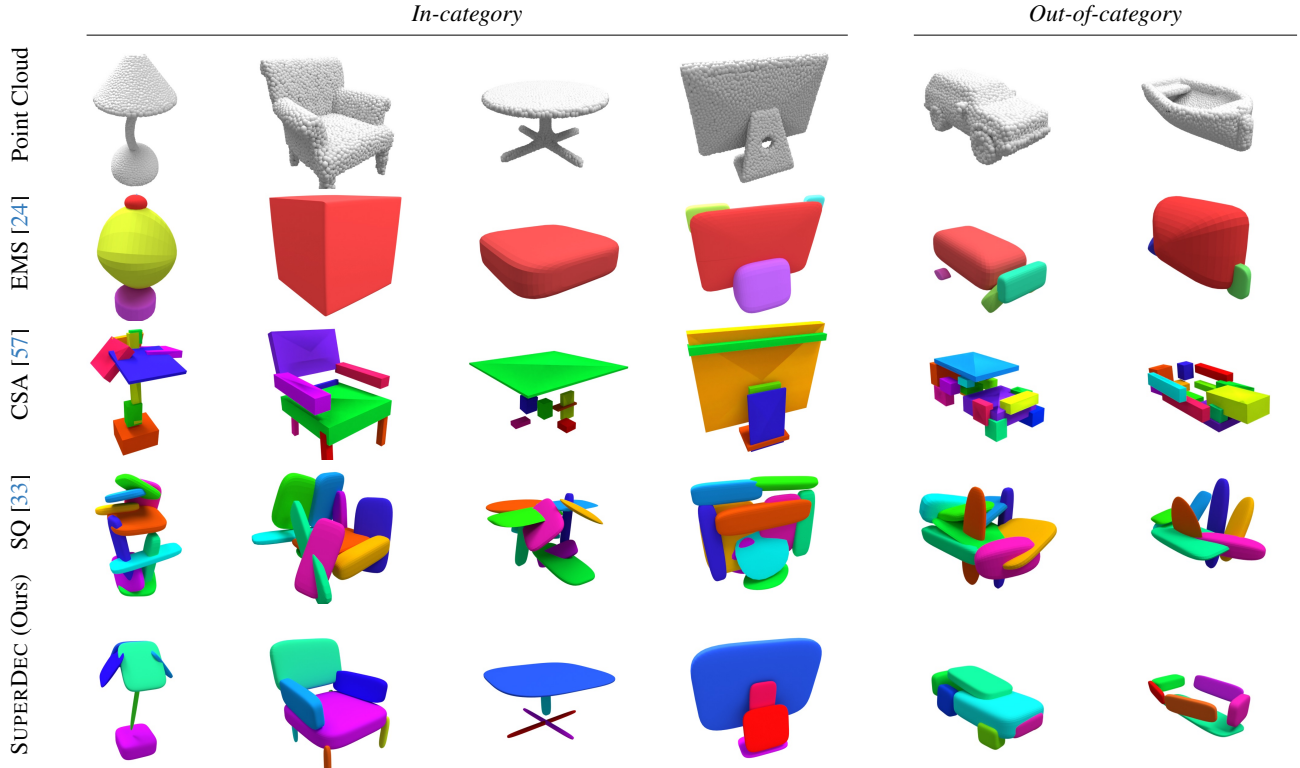


Figure 3. **Qualitative Results on ShapeNet [4]**. We show results on test samples for in-category (*four first columns*) classes and out-of-category classes (*two last columns*). The latter were not seen during training and illustrate how well models generalize to novel classes.

served, and subject to random orientation and translation, providing a more realistic and challenging evaluation setting for our method.

Replica [45]: Finally, we present both quantitative and qualitative results on full 3D scenes from Replica. Object instances are extracted using either ground truth instance annotations or the pre-trained 3D instance segmentation model Mask3D [41], enabling us to evaluate our approach in a fully realistic setting, including scenarios where no ground truth annotations are available.

Methods in comparison. We compare to learning- and optimization-based prior works using both cuboids and superquadrics as geometric primitives. *SQ* [33] is a learning-based approach for object-level decomposition using superquadrics; it takes a voxel grid as input and predicts superquadric primitives via a CNN. *CSA* [57] is another learning-based method but uses cuboids as geometric prim-

itives. It takes a point cloud as input and predicts cuboid parameters from a global latent code. Lastly, *EMS* [24] is an optimization-based approach that decomposes objects by hierarchically fitting superquadrics to parts of a point cloud.

Training Details. Our goal is to develop a general-purpose, class-agnostic model capable of representing arbitrary objects as superquadrics. Existing methods typically train separate models for each object class, assuming that all classes are known in advance and that sufficient training data is available for each. These assumptions, however, often fail in real-world scenarios. To address this, we jointly train a single model on all 13 ShapeNet classes using the publicly available code of prior methods, moving towards a more realistic *class-agnostic* solution. In our model we set the following hyper-parameters $P=16$, $S=4096$, $K=25$, $H=128$, $D=3$, $\epsilon_{exist}=24$, $\lambda_{exist}=0.01$, $\lambda_{par}=0.06$. We refer to the supplementary for additional details.

Method	ScanNet++ [58]			Replica [45]		
	L1 ↓	L2 ↓	#Prim. ↓	L1 ↓	L2 ↓	#Prim. ↓
SQ [33]	10.45	4.26	10.0	11.84	5.91	10
EMS [24]	5.51	2.11	4.25	5.40	2.12	3.61
CSA [57]	2.91	0.41	11.64	3.68	0.70	9.63
SUPERDEC (Ours)	1.70	0.11	5.18	1.79	0.19	6.58

Table 2. **Quantitative Results on Scenes.** Evaluation on objects from 3D scene datasets [45, 58]. Scores are scaled by 10^2 .

Metrics. We assess *reconstruction accuracy* using L1 and L2 Chamfer distances and *compactness* by the average number of geometric primitives.

4.1.1. Results on ShapeNet

We show scores in Tab. 1 and qualitative results in Fig. 3. To evaluate both accuracy and generalization, we conduct two experiments: *in-category* and *out-of-category*. In the *in-category* setting, all learning-based methods are jointly trained on the 13 classes of the ShapeNet training set and evaluated on the corresponding test set. In the *out-of-category* setting, models are trained on half of the categories (*airplane, bench, chair, lamp, rifle, table*) and tested on the remaining ones (*car, sofa, loudspeaker, cabinet, display, telephone, watercraft*). Our SUPERDEC model significantly outperforms both learned and non-learned baselines. Compared to learned baselines, we reduce the L2 loss by a factor of six while using nearly half the number of primitives, supporting our hypothesis that leveraging local point features as opposed to a single global descriptor improves 3D decomposition in both accuracy and compactness. Compared to the non-learned baseline, we predict a similar number of primitives but achieve an L2 loss approximately 20 times smaller, validating the benefit of learning shape priors to avoid local minima that often hinder purely optimization-based approaches.

4.1.2. Results on 3D Scenes

In this section, models are evaluated on real-world, out-of-category objects, which appear in arbitrary orientations and often exhibit incomplete point clouds due to reconstruction artifacts and occlusions. Tab. 2 shows the quantitative results. Despite never being trained on real-world objects, our method outperforms both the optimization- and the learning-based baselines by a large margin. Lastly, we qualitatively evaluate our pipeline on full 3D scenes from Replica, where our object-level model is applied on top of class-agnostic instance segmentation predictions from Mask3D [41]. As shown in Fig. 1, our method effectively reconstructs object shapes, even under noisy segmentation masks and geometries that differ substantially from those seen during training. We refer to the supplementary for additional qualitative results.

Method	Time (ms)	Suc. (%)	Mem. (MB)
Occupancy	0.056	100.00	0.873
PointCloud	0.063	89.57	19.286
Voxels	0.030	98.78	0.101
Cuboids [38]	0.120	61.23	0.024
SUPERDEC	0.150	91.71	0.042

Table 3. **Path Planning Scores.** Mean on 15 ScanNet++ scenes.



Figure 4. **Grasping Result.** Visualization of computed grasp poses for a **milk bottle**, some **flowers**, a **side table**, and a **plant**.

4.2. Down-stream Applications

Next, we show the versatility of the SUPERDEC representation for downstream applications, including robotics tasks such as path planning and object grasping (Sec. 4.2.1), and controllable image generation (Sec. 4.2.2).

4.2.1. Robotics

Path planning seeks to compute a collision-free shortest path between a given start and end point in 3D space, enabling efficient robot navigation. Although essential for traversing large environments, it typically demands storing large-scale 3D representations. Here, we assess whether our compact representation can perform this task effectively while reducing memory requirements. We conduct experiments on 15 ScanNet++ [58] scenes, comparing SUPERDEC to common 3D representations, including dense occupancy grids, point clouds, voxel grids, and cuboids [38]. As shown in Tab. 6, SUPERDEC not only reduces memory consumption compared to traditional representations but also achieves a higher success rate than dense point clouds. Further details about experiment setup, metrics and analysis are provided in the Appendix.

Object Grasping enables robots to grasp real-world objects by computing suitable grasping poses. Existing methods fall into two categories, each with complementary limitations. Geometry-based approaches [3, 12, 31] require precise 3D object models, which are often unavailable in real-world scenarios. Learning-based approaches [28, 47, 56] operate directly on raw sensor data but tend to be biased towards training data, which typically consists of tabletop scenes with small, convex, or low-genus objects [11, 46]. To overcome these limitations, SuperQ-GRASP [50] explored decomposing objects into explicit primitives. How-



Figure 5. **Real-world robot experiment.** The top row shows the input scan (left) and the representation from SUPERDEC with the computed path and grasping pose (right). The bottom row illustrates the robot following the planned path. We denote the starting point of the path with a green sphere, and the target location with a red sphere. The target object (a milk bottle) is circled in red.

ever, its reliance on Marching Primitives [25] to obtain superquadrics from the object’s Signed Distance Function (SDF) makes it unsuitable for most real-world cases where only point clouds are available. In contrast, our approach directly processes point clouds of entire scenes and, when combined with the class-agnostic segmentations from Mask3D [41], extracts superquadrics for all objects. Given the superquadric parameters, we employ a superquadric-based geometric method [55] to compute grasping poses for selected objects. Fig. 4 shows predicted grasping poses on objects from a real-world 3D scan of a room. In practice, our method eliminates the need for data-driven grasping models while remaining adaptable to diverse object shapes and producing high-quality grasping poses.

Real-world Experiment. Finally, we demonstrate the real-world applicability of our superquadric-based representation by deploying it on a legged robot (Boston Dynamics Spot) equipped with an arm, supporting both motion planning and object grasping in an indoor environment. We scan a scene using a 3D scanning application on an iPad, extract a dense point cloud, and run SUPERDEC on it. Given the robot’s starting position and a specified target object (a milk bottle), we compute both the path and the grasping pose as described earlier (see Fig. 4), enabling the robot to approach and successfully grasp the object. Fig. 5 shows the computed representation, the planned trajectory, the grasping pose, and a frame from the real-world demonstration. This experiment suggests that integrating SUPERDEC with open-vocabulary segmentation methods such as OpenMask3D [48] could allow robots to navigate to and grasp arbitrary objects specified via natural-language prompts.

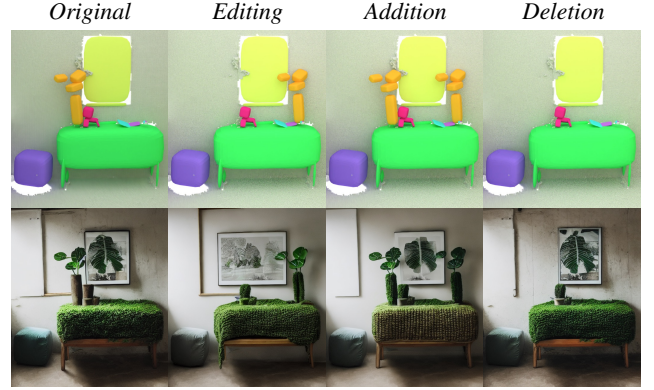


Figure 6. **Spatial control using SUPERDEC.** Top row shows superquadrics generated by SUPERDEC, bottom row shows generated images using the prompt ‘A corner of a room with a plant’.

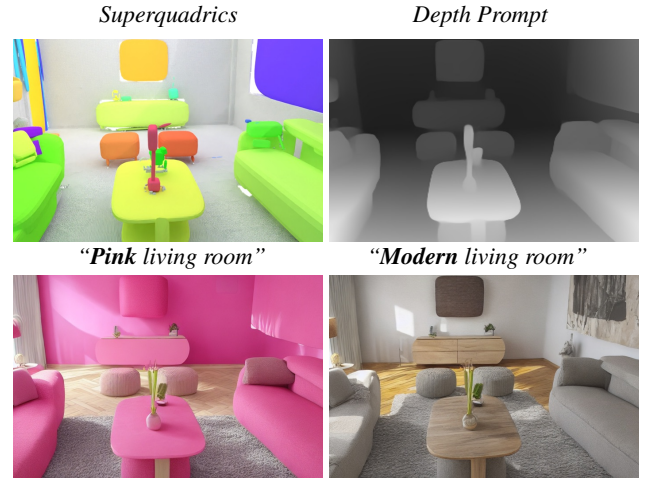


Figure 7. **Semantic control using SUPERDEC.** Top row: superquadrics created by our SUPERDEC, and depth map to prompt the generation of text-to-image diffusion model. Bottom row: generations with two different textual prompts.

4.2.2. Controllable Generation and Editing

We investigate how the SUPERDEC representation can be used to introduce joint spatial and semantic control in text-to-image diffusion models [40]. Specifically, we generate images by conditioning ControlNet [60] on depth maps rendered from the superquadrics extracted from Replica [45] scenes. Qualitative results are shown in Fig. 6 and Fig. 7. Fig. 6 demonstrates spatial control: by *moving*, *duplicating*, or *removing* superquadrics corresponding to a plant, we coherently influence the generated images. Fig. 7 highlights semantic control: we can vary the room’s style while preserving its semantic and geometric structure, and observe that object semantics naturally emerge from the spatial arrangement of superquadrics without explicit conditioning, e.g., pillows appear on couches, and a plant is placed on a central table, reflecting plausible real-world arrangements.

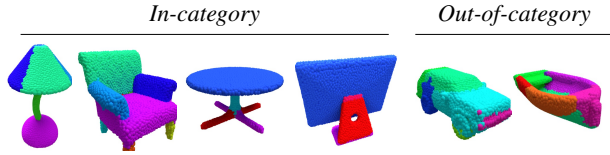


Figure 8. **Qualitative Results on ShapeNet [4] segmentation.** We show the resulting segmentation matrices on test samples for in-category (*four first columns*) classes and out-of-category classes (*two last columns*). The latter were not seen during training.

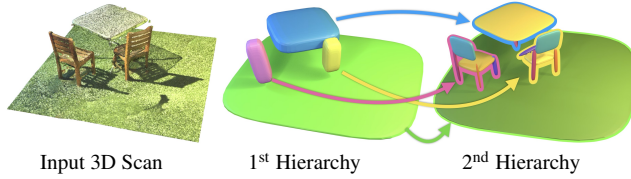


Figure 9. **Hierarchical decomposition with SUPERDEC.** From left to right we visualize: the input point cpngcloud, the superquadrics predicted in the first hierarchy, the superquadrics predicted in the second hierarchy.

4.3. Analysis Experiments

Unsupervised part segmentation. Besides superquadric parameters, our method also predicts a segmentation matrix which segments the initial point cloud into parts that are fitted to the predicted superquadrics. In Fig. 8, we visualize the predicted segmentation masks for the same examples shown in Fig. 3. We observe that segmentation masks, especially in the *in-category* experiments, appear very sharp. This suggests that our method, especially if trained at a larger scale, can be leveraged for different applications as geometry-based part segmentation or as pretraining for supervised semantic part segmentation.

Can we do hierarchical decomposition? We run SUPERDEC in a hierarchical fashion on a whole simple scene, without using any instance segmentation method to segment the scene into different objects. We observed that by applying SUPERDEC on the whole point cloud it can coarsely segment the scene into components such as grass, table, and chairs. By applying the model again on the points segmented during the first prediction, we obtained a more accurate decomposition of the individual object instances. Given that SUPERDEC was trained only on single object instances from ShapeNet [4] we found this results very interesting. We see future potential in the use of our model to obtain unsupervised hierarchical decompositions.

What does our network learn? Since our network performs unsupervised part segmentation, we analyze the features learned by the Transformer decoder across object classes. Inspired by BERT [10]’s [CLS] token, we append a learnable embedding to the sequence of embedded

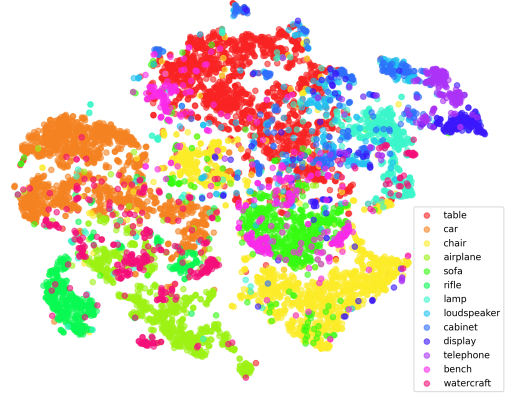


Figure 10. **t-SNE Visualization of Primitive Embeddings** across different ShapeNet classes.

superquadrics; although never explicitly decoded, this embedding is refined through self- and cross-attention. After training, we extract and visualize these embeddings using t-SNE [52] for ShapeNet [4] categories (Fig. 10). We observe that categories with consistent shapes, such as *chairs*, *airplanes*, and *cars*, form clear clusters, while categories with high intra-class variability, such as *watercraft*, are more dispersed. This indicates that our model organizes objects by geometric structure without requiring class annotations.

How fast is our method? Our model is highly parallelizable, allowing multiple objects to be batched and processed simultaneously in a single forward pass. On an RTX 4090 (24 GB), we can process up to 256 objects in parallel. On average, the forward pass takes 0.13 s for a complete Replica [45] scene, 3D instance segmentation with Mask3D [41] requires 0.3 s, and each LM optimization step takes less than 1 s, see supplementary for more details.

5. Conclusion

We proposed SUPERDEC, a method for deriving compact yet expressive 3D scene representations based on simple geometric primitives – specifically, superquadrics. Our model outperforms prior primitive-based methods and generalizes well to out-of-category classes. We further demonstrated the potential of the resulting 3D scene representation for various applications in robotics, and as a geometric prompt for diffusion-based image generation. While this is only a first step towards more compact, geometry-aware 3D scene representations, we anticipate broader applications and expect to see further research in this direction.

Acknowledgments. Elisabetta Fedele is supported by the ETH AI Center doctoral fellowship, by the Swiss National Science Foundation (SNSF) Advanced Grant 216260 (*Beyond Frozen Worlds: Capturing Functional 3D Digital Twins from the Real World*), and an SNSF Mobility Grant. Francis Engelmann is supported by an SNSF PostDoc.mobility grant. We also gratefully acknowledge a GPU grant from NVIDIA.

References

- [1] Stephan Alaniz, Massimiliano Mancini, and Zeynep Akata. Iterative superquadric recomposition of 3d objects from multiple viewsd. In *International Conference on Computer Vision (ICCV)*, 2023. 1
- [2] Alan H. Barr. Superquadrics and angle-preserving transformations. *IEEE Computer Graphics and Applications*, 1981. 2
- [3] Junhao Cai, Jingcheng Su, Zida Zhou, Hui Cheng, Qifeng Chen, and Michael Yu Wang. Volumetric-based contact point detection for 7-dof grasping. In *Conference on Robot Learning (CoRL)*, 2022. 6
- [4] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical report, 2015. 2, 4, 5, 8
- [5] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention Mask Transformer for Universal Image Segmentation. *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2, 3
- [6] Bowen Cheng, Alexander G. Schwing, and Alexander Kirillov. Per-Pixel Classification is Not All You Need for Semantic Segmentation. In *International Conference on Neural Information Processing Systems (NeurIPS)*, 2021. 2, 3
- [7] Laurent Chevalier, Fabrice Jaillet, and Atila Baskurt. Segmentation and superquadric modeling of 3d objects. In *International Conference in Central Europe on Computer Graphics and Visualization*, 2003. 2
- [8] Christopher B Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3D-R2N2: A Unified Approach for Single and Multi-view 3D Object Reconstruction. In *European Conference on Computer Vision (ECCV)*, 2016. 4
- [9] Alexandros Delitzas, Ayca Takmaz, Federico Tombari, Robert Sumner, Marc Pollefeys, and Francis Engelmann. SceneFun3D: Fine-Grained Functionality and Affordance Understanding in 3D Scenes. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies.*, 2019. 8
- [11] Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, and Cewu Lu. Anygrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Transactions on Robotics*, 2023. 6
- [12] B. Faverjon and J. Ponce. On Computing Two-finger Force-closure Grasps of Curved 2D Objects. In *International Conference on Robotics and Automation (ICRA)*, 1991. 6
- [13] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *International Conference on Robotics and Automation (ICRA)*, 2024. 1
- [14] Abdullah Hamdi, Luke Melas-Kyriazi, Jinjie Mai, Guocheng Qian, Ruoshi Liu, Carl Vondrick, Bernard Ghanem, and Andrea Vedaldi. Ges: Generalized exponential splatting for efficient radiance field rendering. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [15] Jan Held, Renaud Vandeghen, Abdullah Hamdi, Adrien Del'ège, Anthony Cioppa, Silvio Giancola, Andrea Vedaldi, Bernard Ghanem, and Marc Van Droogenbroeck. 3D convex splatting: Radiance field rendering with 3D smooth convexes. *ArXiv*, 2024. 2
- [16] Rui Huang, Guangyao Zhai, Zuria Bauer, Marc Pollefeys, Federico Tombari, Leonidas Guibas, Gao Huang, and Francis Engelmann. Video perception models for 3d scene synthesis. *arXiv preprint arXiv:2506.20601*, 2025. 1
- [17] Ales Jaklic, Ales Leonardis, and Franc Solina. *Segmentation and recovery of superquadrics*. Springer Science & Business Media, 2000. 3, 1
- [18] Rasmus Ramsbøl Jensen, A. Dahl, George Vogiatzis, Engil Tola, and Henrik Aanæs. Large scale multi-view stereopsis evaluation. *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014. 2
- [19] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions On Graphics (TOG)*, 2023. 1, 2
- [20] Sebastian Koch, Narunas Vaskevicius, Mirco Colosi, Pedro Hermosilla, and Timo Ropinski. Open3DSG: Open-vocabulary 3D Scene Graphs from Point Clouds with Queryable Objects and Open-set Relationships. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1
- [21] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 1944. 3, 4
- [22] Lei Li and Angela Dai. Genzi: Zero-shot 3d human-scene interaction generation. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1
- [23] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. Soft rasterizer: A differentiable renderer for image-based 3d reasoning. *International Conference on Computer Vision (ICCV)*, 2019. 2
- [24] Weixiao Liu, Yuwei Wu, Sipu Ruan, and Gregory S Chirikjian. Robust and Accurate Superquadric Recovery: A Probabilistic Approach. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 1, 2, 5, 6
- [25] Weixiao Liu, Yuwei Wu, Sipu Ruan, and Gregory S Chirikjian. Marching-primitives: Shape abstraction from signed distance function. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2, 7
- [26] Zhijian Liu, Haotian Tang, Yujun Lin, and Song Han. Point-Voxel CNN for Efficient 3D Deep Learning. In *Inter-*

- tional Conference on Neural Information Processing Systems (NeurIPS), 2019. 3
- [27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019. 1
- [28] Jeffrey Mahler, Matthew Matl, Vishal Satish, Michael Danielczuk, Bill DeRose, Stephen McKinley, and Ken Goldberg. Learning ambidextrous robot grasping policies. *Science Robotics*, 2019. 6
- [29] Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of The Society for Industrial and Applied Mathematics*, 1963. 3, 4
- [30] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020. 1
- [31] A.T. Miller, S. Knoop, H.I. Christensen, and P.K. Allen. Automatic Grasp Planning Using Shape Primitives. In *International Conference on Robotics and Automation (ICRA)*, 2003. 6
- [32] Tom Monnier, Jake Austin, Angjoo Kanazawa, Alexei A. Efros, and Mathieu Aubry. Differentiable blocks world: Qualitative 3d decomposition by rendering primitives. *International Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 1, 2
- [33] Despoina Paschalidou, Ali Osman Ulusoy, and Andreas Geiger. Superquadrics revisited: Learning 3d shape parsing beyond cuboids. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 1, 2, 5, 6
- [34] Songyou Peng, Kyle Genova, Chiyu Max Jiang, Andrea Tagliasacchi, Marc Pollefeys, and Thomas Funkhouser. OpenScene: 3D Scene Understanding with Open Vocabularies. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 1
- [35] Alex P Pentland. Parts: structured descriptions of shape. In *Proceedings of the Fifth AAAI National Conference on Artificial Intelligence*, 1986. 2
- [36] Maurizio Pilu and Robert B Fisher. Equal-distance sampling of superellipse models. In *British Machine Vision Conference (BMVC)*, 1995. 3
- [37] Charles R Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *International Conference on Neural Information Processing Systems (NeurIPS)*, 2017. 4
- [38] Michaël Ramamonjisoa, Sinisa Stekovic, and Vincent Lepetit. MonteBoxFinder: Detecting and Filtering Primitives to Fit a Noisy Point Cloud. In *European Conference on Computer Vision (ECCV)*, 2022. 6, 5
- [39] Aaron Ray, Christopher Bradley, Luca Carlone, and Nicholas Roy. Task and motion planning in hierarchical 3d scene graphs. *arXiv preprint arXiv:2403.08094*, 2024. 1
- [40] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 7
- [41] Jonas Schult, Francis Engelmann, Alexander Hermans, Or Litany, Siyu Tang, and Bastian Leibe. Mask3d: Mask transformer for 3d semantic instance segmentation. *International Conference on Robotics and Automation (ICRA)*, 2022. 2, 3, 4, 5, 6, 7, 8
- [42] Jonas Schult, Sam Tsai, Lukas Höllein, Bichen Wu, Jialiang Wang, Chih-Yao Ma, Kunpeng Li, Xiaofang Wang, Felix Wimbauer, Zijian He, et al. Controlroom3d: Room generation using semantic proxy rooms. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1
- [43] Jaidev Shriram, Alex Trevithick, Lingjie Liu, and Ravi Ramamoorthi. Realdreamer: Text-driven 3d scene generation with inpainting and depth diffusion. *arXiv preprint arXiv:2404.07199*, 2024. 1
- [44] Franc Solina and Ruzena Bajcsy. Recovery of parametric models from range images: The case for superquadrics with global deformations. *Transactions on Pattern Analysis and Machine Intelligence (PAMI)*, 1990. 2
- [45] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Yuheng Ren, Shobhit Verma, Anton Clarkson, Ming Yan, Brian Budge, Yajie Yan, Xiaqing Pan, June Yon, Yuyang Zou, Kimberly Leon, Nigel Carter, Jesus Briales, Tyler Gillingham, Elias Mueggler, Luis Pesqueira, Manolis Savva, Dhruv Batra, Hauke Malte Strasdat, Renzo De Nardi, Michael Goesele, S. Lovegrove, and Richard A. Newcombe. The replica dataset: A digital replica of indoor spaces. *ArXiv*, 2019. 2, 4, 5, 6, 7, 8, 1, 3
- [46] Matan Sudry, Tom Jurgenson, Aviv Tamar, and Erez Karpas. Hierarchical planning for rope manipulation using knot theory and a learned inverse model. In *Conference on Robot Learning*, 2023. 6
- [47] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes. In *International Conference on Robotics and Automation (ICRA)*, 2021. 6
- [48] Ayça Takmaz, Elisabetta Fedele, Robert W. Sumner, Marc Pollefeys, Federico Tombari, and Francis Engelmann. OpenMask3D: Open-Vocabulary 3D Instance Segmentation. In *International Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 1, 7
- [49] Ayca Takmaz, Alexandros Delitzas, Robert W. Sumner, Francis Engelmann, Johanna Wald, and Federico Tombari. Search3D: Hierarchical Open-Vocabulary 3D Segmentation. *IEEE Robotics and Automation Letters (RA-L)*, 2025. 1
- [50] Xun Tu and Karthik Desingh. Superq-grasp: Superquadrics-based grasp pose estimation on larger objects for mobile-manipulation. *arXiv*, 2024. 6
- [51] Shubham Tulsiani, Hao Su, Leonidas J. Guibas, Alexei A. Efros, and Jitendra Malik. Learning shape abstractions by assembling volumetric primitives. *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 1, 2
- [52] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of machine learning research*, 2008. 8

- [53] Erik Roeland van Dop and Paulus P.L. Regtien. Fitting undeformed superquadrics to range data: improving model recovery and classification. *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 1998. [3](#)
- [54] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *International Conference on Neural Information Processing Systems (NeurIPS)*, 2017. [3](#)
- [55] Giulia Vezzani, Ugo Pattacini, and Lorenzo Natale. A grasping approach based on superquadric models. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1579–1586. IEEE, 2017. [7](#)
- [56] Chenxi Wang, Hao-Shu Fang, Minghao Gou, Hongjie Fang, Jin Gao, and Cewu Lu. Graspness Discovery in Clutters for Fast and Accurate Grasp Detection. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. [6](#)
- [57] Kaizhi Yang and Xuejin Chen. Unsupervised learning for cuboid shape abstraction via joint segmentation from point clouds. *ACM Transactions On Graphics (TOG)*, 2021. [1](#), [2](#), [3](#), [5](#), [6](#)
- [58] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. *International Conference on Computer Vision (ICCV)*, 2023. [2](#), [4](#), [6](#), [1](#), [3](#), [5](#)
- [59] Chenyangguang Zhang, Alexandros Delitzas, Fangjinhua Wang, Ruida Zhang, Xiangyang Ji, Marc Pollefeys, and Francis Engelmann. Open-Vocabulary Functional 3D Scene Graphs for Real-World Indoor Spaces. In *International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. [1](#)
- [60] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. *International Conference on Computer Vision (ICCV)*, 2023. [7](#)

SUPERDEC: 3D Scene Decomposition with Superquadric Primitives

Supplementary Material

1. Superquadrics

While our architecture can be easily adapted to segment and predict in an unsupervised manner other types of geometric primitives - in SUPERDEC we decided to use superquadrics. When looking for a suitable geometric primitive for our approach we were keeping in mind two main criteria. First, we wanted the primitive to be represented by a compact parameterization so that it can be described by only using a few parameters. Second, we wanted the representation to be expressive, in order to be able to describe real-world objects by only using a few primitives. Inspired by 3DGS [19], the first parameterization we took into consideration were the *ellipsoids*. Ellipsoids have a very compact parameterization as their shape can be represented using the following implicit equation:

$$f(\mathbf{x}) = \left(\frac{x}{s_x}\right)^2 + \left(\frac{y}{s_y}\right)^2 + \left(\frac{z}{s_z}\right)^2 = 1,$$

where the only free variables are s_x , s_y , s_z , which are the lengths of the three main semi-axis. However, if we start thinking about which objects and object parts can be effectively fitted using a single ellipsoid, we realize that their representational capabilities are not enough. In order to obtain higher representational capabilities while still keeping a simple representation, a natural extension are *generalized ellipsoids*. In this representation, we not only allow the length of the semi-axis to be variable, but their roundness controlled by the three exponents, which previously were fixed to 2. In that way, we obtain the following implicit function:

$$f(\mathbf{x}) = \left(\frac{|x|}{s_x}\right)^{e_1} + \left(\frac{|y|}{s_y}\right)^{e_2} + \left(\frac{|z|}{s_z}\right)^{e_3} = 1.$$

Using generalized ellipsoids with high exponents it becomes possible to also represent cuboidal shapes. While having suitable representational capabilities, these primitives do not allow to compute distance to their surface in a closed form, a property which can be extremely useful for various downstream applications. This drawback is overcome by *superquadrics*, at the cost of one less degree of freedom, which however does not substantially impact expressivity. Unlike generalized ellipsoids, which assign a separate roundness parameter to each axis, superquadrics share the same roundness for the x and y axes while allowing a distinct parameter for the z axis. Their shape is

represented in implicit form by the equation:

$$f(\mathbf{x}) = \left(\left(\frac{x}{s_x}\right)^{\frac{2}{e_2}} + \left(\frac{y}{s_y}\right)^{\frac{2}{e_2}}\right)^{\frac{e_2}{e_1}} + \left(\frac{z}{s_z}\right)^{\frac{2}{e_1}} = 1,$$

and the euclidean radial distance to their surface can be computed in closed form, as shown in Eq. 2. In addition, superquadrics are also equipped with an explicit function, which can be used to sample points from their surface. Specifically, given the coordinates (η, ω) such that $\eta \in [-\frac{\pi}{2}, \frac{\pi}{2}]$ and $\omega \in [-\pi, \pi]$, the surface of a superquadric can be represented as:

$$s(\eta, \omega) = \begin{bmatrix} s_x \cos(\eta)^{e_1} \cos(\omega)^{e_2} \\ s_y \cos(\eta)^{e_1} \sin(\omega)^{e_2} \\ s_z \sin(\eta)^{e_1} \end{bmatrix}.$$

We refer to [17] for additional details on superquadrics.

2. Training Details

In general, we train our model for 500 epochs with $\lambda_{par} = 0.1$ and then for other 500 epochs with $\lambda_{par} = 0.6$. For the results on Replica [45] and ScanNet++ [58] we trained our model on normalized objects (centered and rescaled to fit in a sphere of radius 0.5). During training we apply rotation and translation augmentations. Specifically, we apply random rotations between 0° and 180° around the z axis and between 0° and 7.5° on the x and y axis. We also apply random translation with respect to the center in a radius of 0.05. We optimize our network with Adam [27] a one-cycle learning rate schedule with a maximum learning rate of $4e-4$. We train on 4 NVIDIA A100 with total batch size of 128.

3. Additional Results

Does LM improve our final predictions? In our approach we use LM optimization as a post processing step. In this experiment (Fig. 11) we want to assess how a different number of LM optimization rounds affects the final predictions in terms of L2 Chamfer Distance. In order to evaluate this aspect, we report L2 loss after different numbers of LM optimization steps, evaluating both in-category and out-of-category. From this experiment we can notice two main aspects. Firstly, we see that it leads to larger improvements in the out-of-category rather than in the in-category one. This is probably due to the less accurate initial predictions of our feedforward model in this setting and it shows that our optimization step can be used to decrease the gap between in-category and out-category. Secondly, we see that even if

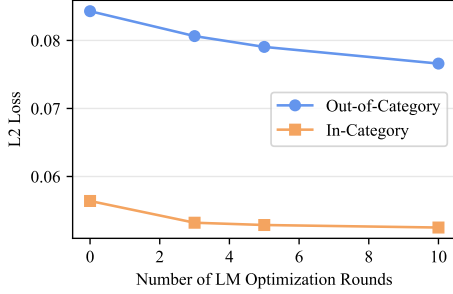


Figure 11. **LM optimization experiment.** We show how LM optimization improves results in terms of L2 Chamfer distance across a variable number of rounds. We report results both for in-category experiments and out-of-category ones.

Method	Training	Inference	L1 ↓	L2 ↓	#Prim. ↓
SQ	-	-	3.668	0.279	10
SUPERDEC (Ours)	✓	✓	1.698	0.047	5.79
SUPERDEC (Ours)	✓	✗	1.695	0.046	5.82
SUPERDEC (Ours)	✗	✗	1.711	0.049	5.84

Table 4. **Ablation study on existence head.** Evaluation on objects from ShapeNet [4] dataset. The first two columns indicate whether we use the predicted α_j at training (left) and inference (right) time. Scores are scaled by 10^2 .

LM optimization improves our final predictions, it does not lead to substantial improvements. This suggests that the solutions predicted by our method are located in local minima and that a diverse type of optimization should be resorted to improve the predictions further.

Is the existence head needed? As explained in the main paper, SUPERDEC predicts a set of P superquadrics and a segmentation matrix. As in many cases less than P superquadrics are in fact needed to represent the input point cloud, we also predict an existence parameter (α_j) for each of them, which allows us to model a variable number of primitives. However, the existence of a superquadric can also be directly deducted from the segmentation matrix, by computing $\hat{\alpha}_j$. We made some experiments to assess the impact of using either α_j or $\hat{\alpha}_j$, both at training and at inference time and reported the results in Tab. 4. In order to evaluate the impact of the prediction head, we conducted two additional experiments. In the first experiment, we evaluate the model trained as explained in Sec. 3.1. In the second experiment, we retrained our model without explicitly supervising α_j and using $\hat{\alpha}_j$ at inference time.

Compactness–Accuracy Trade-off. The hyperparameter λ_{par} controls the trade-off between reconstruction accuracy and representation compactness (see Eq. 4). We evaluate this trade-off quantitatively by first training the model with $\lambda_{par} = 0.1$ for 500 epochs, followed by fine-tuning for 100 epochs with varying λ_{par} values. Fig. 12 shows the im-

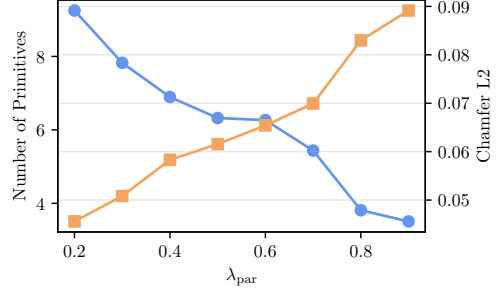


Figure 12. **Compactness vs. reconstruction accuracy tradeoff.** We run experiment for different values of the parsimony weight λ_{par} (x-axis) and we visualize the resulting number of primitives (y-axis, left) and the L2 Chamfer distance (y-axis, right).

Method	ScanNet++ [58]				Replica [45]		
	FPS	L1 ↓	L2 ↓	#Prim. ↓	L1 ↓	L2 ↓	#Prim. ↓
CSA	✓	2.91	0.41	11.64	3.68	0.70	9.63
SUPERDEC (Ours)	✓	1.70	0.11	5.18	1.79	0.19	6.58
SUPERDEC (Ours)	✗	1.74	0.13	5.19	1.81	0.20	6.52

Table 5. **Ablation study on sampling technique.** Evaluation on objects from 3D scene datasets [45, 58]. The second column report whether Farthest Point Sampling (FPS) was used to sample the input points. Alternatively, points were randomly sampled. Scores are scaled by 10^2 .

part of λ_{par} on Chamfer distance and the average number of predicted primitives. By adjusting λ_{par} , the model can smoothly balance compactness and accuracy, allowing for easy fine-tuning to meet target reconstruction quality. In our experiments, we use $\lambda_{par} = 0.6$, which approximately corresponds to the intersection point of the two curves.

Is FPS needed? To understand whether point clouds need to be downsampled with Farthest Point Sampling (FPS) or points can just be randomly sampled, we compare the performances of SUPERDEC on 3D scene datasets using the two approaches. We observe that the performances do not degenerate and that random sampling is enough for practical applications.

4. Qualitative results on scenes

4.1. Replica

In Fig. 13 we visualize the superquadric decompositions for some scenes from Replica [45]. The instance masks are predicted using Mask3D [41]. The points not belonging to any instance are visualized in gray.

4.2. ScanNet++

In Fig. 14 we visualize the superquadric decompositions for some scenes from ScanNet++ [58]. The instance masks the ground truth instance masks. The points not belonging to any instance are visualized in gray.



Figure 13. **Qualitative Results on Replica [45].** We show results on different scenes from Replica. In the first row we show a rendering of the input mesh, while on the second row we show the output superquadrics. We visualize different object instances with different colors.



Figure 14. **Qualitative Results on ScanNet++ [58].** We show results on different scenes from ScanNet++. In the first row we show a rendering of the input mesh, while on the second row we show the output superquadrics. We visualize different object instances with different colors.

5. Robot Experiment

In this section we introduce the key methods and parameters used in our robot experiments. We also present more detailed qualitative and quantitative evaluation results.

5.1. Setup

For *path planning* in both ScanNet++ [58] and real-world scenarios, we use the Python binding of the Open Motion Planning Library (OMPL). The state space is defined as a *3D RealVectorStateSpace*, with boundaries extracted from

the 3D bounding box of the input point cloud. We employ a sampling-based planner (RRT*), setting a maximum planning time of 2 seconds per start-goal pair.

In ScanNet++ scenes, the occupancy grid and voxel grid are both set to a 10 cm resolution, with voxels generated from the original point cloud. The collision radius is 25 cm. For dense occupancy grid planning, we enforce an additional constraint in the validity checking to ensure that paths remain within 25 cm of free space, preventing them from extending outside the scene or penetrating walls. And the planned occupancy grid path serves as a reference for

computing relative path optimality in our evaluation. Start and goal points are sampled within a 0.4m-0.6m height range in free space, as most furniture and objects are within this range. This allows for a fair evaluation of how different representations capture collisions for valid path planning. During evaluation, we further validate paths by interpolating them into 5 cm waypoint intervals. Each waypoint is checked against the occupancy grid to ensure that its nearest occupied grid is beyond 25 cm and its nearest free grid is within 25 cm. A path is considered unsuccessful if more than 10% of waypoints fail this check. This soft constraint accounts for the sampling-based nature of RRT*, which does not enforce voxel-level validity but instead checks waypoints along the tree structure, leading to occasional minor violations. In the real-world path planning, we set the collision radius to 60 cm to approximate the size of the Boston Dynamics Spot robot. Spot follows the planned path using its Python API for execution. For *grasping* in real-world experiments, we use the [superquadric-library](#) to compute single-hand grasping poses based on superquadric parameters. The process begins by identifying the object of interest and its corresponding superquadric decomposition. One of the superquadrics is selected and fed into the grasping estimator. To execute the grasp, the robot first navigates to the object’s location during the planning stage. Then, using its built-in inverse kinematics planner and controller, the robot moves its end-effector to the estimated grasping pose for object manipulation.

5.2. Planning Results

In Tab. 6 we report the complete planning results on 15 Scannet++ [58] scenes.

Method	0a76e06478				0c6c7145ba				0f0191b10b				1a8e0d78e0				1a130d092a			
	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.
Occupancy	0.05	100	1.00	960KB	0.06	100	1.00	667KB	0.06	100	1.00	1031KB	0.05	100	1.00	926KB	0.05	100	1.00	803KB
PointCloud	0.07	86	0.98	18MB	0.09	91	0.99	12MB	0.03	77	0.99	19MB	0.05	91	0.99	18MB	0.05	89	0.98	18MB
Voxels	0.03	100	0.97	91KB	0.03	100	1.00	65KB	0.03	100	0.99	99KB	0.03	100	1.01	91KB	0.03	100	1.09	99KB
Cuboids [38]	0.11	32	0.98	22KB	0.10	18	1.02	19KB	0.14	85	1.03	34KB	0.10	50	1.06	21KB	0.12	79	1.00	27KB
SUPERDEC	0.17	100	0.99	52KB	0.16	100	0.97	48KB	0.17	92	0.94	51KB	0.14	91	0.99	39KB	0.13	100	0.98	35KB
Method	0a76e06478				0b031f3119				0dce89ab21				0e350246d4				0eba3981c9			
	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.
Occupancy	0.05	100	1.00	916KB	0.06	100	1.00	1760KB	0.05	100	1.00	1070KB	0.06	100	1.00	366KB	0.06	100	1.00	473KB
PointCloud	0.05	86	1.02	18MB	0.06	96	1.04	25MB	0.05	84	1.13	19MB	0.06	88	1.22	10MB	0.14	80	0.98	45MB
Voxels	0.03	100	1.01	99KB	0.03	100	1.00	160KB	0.03	100	1.19	104KB	0.03	100	1.00	51KB	0.03	100	1.12	199KB
Cuboid[38]	0.14	71	1.12	32KB	0.11	78	1.03	24KB	0.11	35	1.00	23KB	0.09	62	1.00	15KB	0.17	87	1.17	41KB
SuperDec	0.16	86	1.17	46KB	0.16	93	0.98	46KB	0.13	100	1.07	33KB	0.15	88	1.22	40KB	0.19	57	1.10	58KB
Method	7cd2ac43b4				1841a0b525				25927bb04c				e0abd740ba				0f25f24a4f			
	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.	Time(ms)	Suc.(%)	Opt.	Mem.
Occupancy	0.06	100	1.00	1241KB	0.05	100	1.00	1053KB	0.06	100	1.00	407KB	0.06	100	1.00	554KB	0.05	100	1	7MB
PointCloud	0.05	100	1.09	25MB	0.04	89	0.98	16MB	0.06	100	1.01	11MB	0.06	97	0.93	16MB	0.07	61	0.97	99MB
Voxels	0.03	100	1.00	137KB	0.03	100	0.98	82KB	0.03	83	1.04	51KB	0.03	100	1.04	83KB	0.03	96	0.96	617KB
Cuboid[38]	0.21	80	1.04	57KB	x	x	x	15KB	0.09	87	0.96	17KB	0.07	52	1.04	11KB	x	x	x	x
SuperDec	0.15	100	1.05	45KB	0.10	94	0.87	18KB	0.17	83	1.30	53KB	0.12	100	0.87	27KB	0.21	57	0.82	71KB

Table 6. **Path Planning Results.** We show results of path planning for different ScanNet++ [58] scenes, whose ids are reported on the top. PointCloud method uses dense point clouds from ScanNet++, all other methods process the same input point cloud. Time refers to average execution time of the validity-check function during the sampling stage of planning. Success rate (Suc.) is calculated after excluding trials where no representation could generate valid path due to randomness of start and goal sampling. The Cuboid method encounters an out-of-memory failure when fitting scene *0f25f24a4f* due to its large scale, and fails to find any valid path in scene *1841a0b525*.